

2.

a. I used Chatgpt and Gemini

b. I used the browser versions on my computer

c. prompt:

Write a C++ program:

The program will prioritize emails for a busy company CEO.

- You will use a MaxHeap as a means of implementing a priority queue. A priority queue is a queue where emails can shift towards the front of the queue based on a priority status.
- You must implement a MaxHeap using a list-based implementation. Then use that MaxHeap to handle all your email prioritizing for the CEO.
- You must create objects from scratch. Do not include pre-existing heap modules.

• Test File format:

Command Description

EMAIL <sender category>,
<subject line>,
<date>

The emails in the CEO's Inbox should be placed in queue based on their sender category and date. The sender categories and priority to be read are as follows:

- Boss – read first
- Subordinate – read next
- Peer – read next
- ImportantPerson – read next
- OtherPerson – read last

If there is more than one from a sender, then the newest email (not the oldest) should be read first. I discovered this trick while a manager at Sprint.

EMAIL is followed by space. The rest of the fields are delimited.

Assume <sender category> is one of the five strings listed above.

Assume <subject line> is a string which may contain spaces, but not commas

Assume <date> is in the format: MM-DD-YYYY

NEXT

Next email for the CEO to read. Display the information on the terminal in the following format:

Sender:

Subject:

Date:

READ CEO has read email and dealt with it

COUNT Display current count of unread emails

- The grader will test your code with a different test file. The test file will begin with the emails in the CEOs Inbox, followed by the commands the CEO will use to read her Inbox. You can assume a given command will be formatted as shown, but the order and number of commands may vary. For example, be able to handle a file that does NEXT or READ when there are no emails, two NEXTs in a row without an intervening READ (output should be the same for both NEXTs), two READs in a row without an intervening NEXT (highest priority email deleted from queue without displaying it).

- Sample Test File:

EMAIL Peer,Can you help me on this?,12-01-2024

EMAIL OtherPerson,Try our product,12-19-2024

EMAIL Boss,Important,12-20-2024

EMAIL Subordinate,How do I handle this?,12-25-2024

EMAIL ImportantPerson,Health Insurance Enrollment,12-31-2024

EMAIL Boss,Never Mind,01-03-2025

COUNT

NEXT

READ

NEXT

READ

COUNT

- Your program should display this:

There are 6 emails to read.

Next email:

Sender: Boss

Subject: Never Mind

Date: 01-03-2025

Next email:

Sender: Boss

Subject: Important

Date: 12-20-2024

There are 4 emails to read.

d.

Chatgpt Initial Code:

```
#include <iostream>
#include <fstream>
#include <sstream>
#include <vector>
```

```

#include <string>
using namespace std;

// -----
// Email object
// -----
class Email {
public:
    string sender;
    string subject;
    string date;

    Email(string s, string sub, string d) {
        sender = s;
        subject = sub;
        date = d;
    }
};

// -----
// MaxHeap class
// -----
class MaxHeap {
private:
    vector<Email> heap;

    // Return the priority of a sender category
    int senderPriority(string sender) {
        if (sender == "Boss")
            return 5;
        else if (sender == "Subordinate")
            return 4;
        else if (sender == "Peer")
            return 3;
        else if (sender == "ImportantPerson")
            return 2;
        else
            return 1;
    }

    // Convert MM-DD-YYYY into a number that can be compared
    int dateValue(string date) {
        int month = stoi(date.substr(0, 2));
        int day = stoi(date.substr(3, 2));
    }
};

```

```

    int year = stoi(date.substr(6, 4));

    return year * 10000 + month * 100 + day;
}

// Returns true if a should have higher priority than b
bool higherPriority(Email a, Email b) {
    int priorityA = senderPriority(a.sender);
    int priorityB = senderPriority(b.sender);

    // Sender category has the highest priority
    if (priorityA != priorityB)
        return priorityA > priorityB;

    // If same sender category, newest email comes first
    return dateValue(a.date) > dateValue(b.date);
}

// -----
// Index helper functions
// -----
int left(int i) {
    return 2 * i + 1;
}

int right(int i) {
    return 2 * i + 2;
}

int parent(int i) {
    return (i - 1) / 2;
}

// -----
// Swap two emails
// -----
void swapEmails(int a, int b) {
    Email temp = heap[a];
    heap[a] = heap[b];
    heap[b] = temp;
}

// -----
// Move an email up the heap

```

```
// -----
void heapifyUp(int i) {
    while (i > 0 && higherPriority(heap[i], heap[parent(i)])) {
        swapEmails(i, parent(i));
        i = parent(i);
    }
}
```

```
// -----
// Move an email down the heap
// -----
void heapifyDown(int i) {
    while (true) {
        int l = left(i);
        int r = right(i);
        int largest = i;

        // Check left child
        if (l < heap.size() &&
            higherPriority(heap[l], heap[largest])) {
            largest = l;
        }

        // Check right child
        if (r < heap.size() &&
            higherPriority(heap[r], heap[largest])) {
            largest = r;
        }

        // If neither child is higher priority, we are done
        if (largest == i)
            break;

        swapEmails(i, largest);
        i = largest;
    }
}
```

public:

```
// -----
// Add email to heap
// -----
void insert(Email email) {
```

```

        heap.push_back(email);
        heapifyUp(heap.size() - 1);
    }

    // -----
    // Remove highest priority email
    // -----
    void removeMax() {
        if (heap.empty())
            return;

        heap[0] = heap.back();
        heap.pop_back();

        if (!heap.empty())
            heapifyDown(0);
    }

    // -----
    // Get highest priority email
    // -----
    Email getMax() {
        return heap[0];
    }

    // -----
    // Check if heap is empty
    // -----
    bool empty() {
        return heap.empty();
    }

    // -----
    // Return number of emails
    // -----
    int size() {
        return heap.size();
    }
};

// -----
// Main program
// -----
int main() {

```

MaxHeap emails;

string line;

// Read input until EOF

while (getline(cin, line)) {

// -----

// EMAIL command

// -----

if (line.substr(0, 5) == "EMAIL") {

// Remove "EMAIL "

string data = line.substr(6);

// Find the commas

int comma1 = data.find(',');

int comma2 = data.find(',', comma1 + 1);

// Extract each field

string sender = data.substr(0, comma1);

string subject = data.substr(
 comma1 + 1,
 comma2 - comma1 - 1

);

string date = data.substr(comma2 + 1);

// Add email to MaxHeap

Email email(sender, subject, date);

emails.insert(email);

}

// -----

// NEXT command

// -----

else if (line == "NEXT") {

if (!emails.empty()) {

Email next = emails.getMax();

cout << "Next email:" << endl;

cout << "Sender: " << next.sender << endl;

cout << "Subject: " << next.subject << endl;

cout << "Date: " << next.date << endl;

```

    }
}

// -----
// READ command
// -----
else if (line == "READ") {

    // Remove highest priority email
    // without displaying it
    if (!emails.empty()) {
        emails.removeMax();
    }
}

// -----
// COUNT command
// -----
else if (line == "COUNT") {

    cout << "There are "
         << emails.size()
         << " emails to read."
         << endl;
}
}

return 0;
}

```

Gemini Initial Code:

```

#include <iostream>
#include <string>
#include <vector>
#include <sstream>

// Representation of an Email
struct Email {
    std::string sender;
    std::string subject;
    std::string date; // MM-DD-YYYY
    int insertionIndex; // Secondary key for stable tracking

```

```

// Helper to map sender categories to numeric priority (higher = read first)
int getCategoryPriority() const {
    if (sender == "Boss") return 5;
    if (sender == "Subordinate") return 4;
    if (sender == "Peer") return 3;
    if (sender == "ImportantPerson") return 2;
    if (sender == "OtherPerson") return 1;
    return 0;
}

// Helper to convert MM-DD-YYYY to YYYYMMDD for strict chronological comparison
int getDateComparable() const {
    if (date.length() < 10) return 0;
    int month = std::stoi(date.substr(0, 2));
    int day = std::stoi(date.substr(3, 2));
    int year = std::stoi(date.substr(6, 4));
    return year * 10000 + month * 100 + day;
}

// Returns true if *this* email has STRICTLY HIGHER priority than *other*
bool operator>(const Email& other) const {
    int p1 = getCategoryPriority();
    int p2 = other.getCategoryPriority();

    if (p1 != p2) {
        return p1 > p2;
    }

    // Same category -> Newest email first (larger YYYYMMDD value)
    int d1 = getDateComparable();
    int d2 = other.getDateComparable();
    if (d1 != d2) {
        return d1 > d2;
    }

    // Tiebreaker for identical category and date -> preserve FIFO/LIFO consistency
    return insertionIndex < other.insertionIndex;
}
};

// Custom MaxHeap implementation built from scratch
class MaxHeap {
private:

```

```
std::vector<Email> heap;
```

```
void heapifyUp(int index) {  
    while (index > 0) {  
        int parent = (index - 1) / 2;  
        // If current node has higher priority than parent, swap  
        if (heap[index] > heap[parent]) {  
            std::swap(heap[index], heap[parent]);  
            index = parent;  
        } else {  
            break;  
        }  
    }  
}
```

```
void heapifyDown(int index) {  
    int size = heap.size();  
    while (true) {  
        int left = 2 * index + 1;  
        int right = 2 * index + 2;  
        int largest = index;  
  
        if (left < size && heap[left] > heap[largest]) {  
            largest = left;  
        }  
        if (right < size && heap[right] > heap[largest]) {  
            largest = right;  
        }  
  
        if (largest != index) {  
            std::swap(heap[index], heap[largest]);  
            index = largest;  
        } else {  
            break;  
        }  
    }  
}
```

```
public:  
    void push(const Email& email) {  
        heap.push_back(email);  
        heapifyUp(heap.size() - 1);  
    }
```

```

void pop() {
    if (heap.empty()) return;
    heap[0] = heap.back();
    heap.pop_back();
    if (!heap.empty()) {
        heapifyDown(0);
    }
}

Email top() const {
    return heap.front();
}

bool empty() const {
    return heap.empty();
}

size_t size() const {
    return heap.size();
}
};

// Main processing logic
int main() {
    MaxHeap emailQueue;
    std::string line;
    int globalInsertionCounter = 0;

    while (std::getline(std::cin, line)) {
        if (line.empty()) continue;

        if (line.rfind("EMAIL ", 0) == 0) {
            std::string payload = line.substr(6);
            std::stringstream ss(payload);

            std::string sender, subject, date;
            std::getline(ss, sender, ',');
            std::getline(ss, subject, ',');
            std::getline(ss, date, ',');

            // Trim leading spaces if any
            if (!sender.empty() && sender[0] == ' ') sender = sender.substr(1);
            if (!subject.empty() && subject[0] == ' ') subject = subject.substr(1);
            if (!date.empty() && date[0] == ' ') date = date.substr(1);

```

```

Email email{sender, subject, date, globalInsertionCounter++};
emailQueue.push(email);

} else if (line == "NEXT") {
    if (!emailQueue.empty()) {
        Email topEmail = emailQueue.top();
        std::cout << "Next email:\n";
        std::cout << "Sender: " << topEmail.sender << "\n";
        std::cout << "Subject: " << topEmail.subject << "\n";
        std::cout << "Date: " << topEmail.date << "\n";
    }
} else if (line == "READ") {
    if (!emailQueue.empty()) {
        emailQueue.pop();
    }
} else if (line == "COUNT") {
    std::cout << "There are " << emailQueue.size() << " emails to read.\n";
}
}

return 0;
}

```

e.

Category	Chatgpt	Gemini
Correctness	Correct on the spec'd input. It has no tiebreak for equal category and date, so the order is arbitrary. <code>line.substr(6)</code> throws on a bare EMAIL line. <code>\r</code> is not stripped.	Correct on the spec'd input. It has a tiebreak, but earlier arrival wins, which arguably contradicts "newest, not oldest". It checks for "EMAIL " with the space, and guards short dates. <code>\r</code> is not stripped.
Execution Time	$O(\log n)$ insert and READ, $O(1)$ NEXT and COUNT. <code>higherPriority(Email a, Email b)</code> takes both emails by value, copying six strings per comparison. <code>swapEmails</code> makes three copies per swap. Output uses <code>endl</code> , which flushes each time.	Same big-O. <code>operator></code> takes <code>const Email&</code> , so there are no copies. It swaps with <code>std::swap</code> , which can move. It outputs <code>"\n"</code> . It still re-runs <code>substr</code> and <code>stoi</code> on every tie-break comparison, and stringstream parsing adds

		overhead.
Space Complexity	$O(n)$ in a vector. Comparisons create temporary copies of the strings, including the subject, so each comparison costs time and temporary space proportional to the string lengths.	$O(n)$. It adds one int (insertionIndex) per email. It has no per-comparison temporaries.
Maintainability	Well commented, with separate Email and MaxHeap classes. The priority and date logic lives inside MaxHeap, not Email. Public data members, using namespace std, and non-const methods.	Comparison logic is encapsulated in Email::operator>, and methods are const-correct. It has fewer comments and a public-member struct.

f.

Gemini because its comparison avoids copying, its ordering logic is encapsulated in Email, it has a deterministic tiebreak, and it is const-correct. Its weaknesses (the oldest-first tiebreak, fewer comments, repeated date parsing) are easy to fix. ChatGPT's per-comparison string copies are a structural performance problem.

g.

- Correctness
 - Change the tiebreak so the later arrival is read first, matching "newest first".
 - Strip `\r` from each line.
 - Replace stoi with a manual digit parse that can't throw.
 - Trim leading and trailing spaces on all fields.
 - Parse with find and rfind instead of stringstream.
- Execution time
 - Compute the priority rank and a YYYYMMDD integer once in the Email constructor.
 - Each heap comparison is then just integer comparisons.
 - `std::move` on insert and on removeMax.
- Space
 - Drop the repeated temporaries.
 - `peek()` returns a pointer instead of a copy.
- Maintainability
 - Private members with accessors.
 - Prologue comments and per-line comments with authorship labels.
 - `size_t` indices, which removes signed/unsigned warnings.